GPU Passthrough for WSL2: CUDA from Inside Ubuntu and Docker

> Take a Windows 11 laptop or desktop with an NVIDIA GPU from "the card sits idle while I'm in WSL" to "PyTorch, JAX, and TensorFlow train on it from Ubuntu — and from inside Dev Containers — at near-native speed." Companion document to scientific-python-2026, which covers the Python data stack that runs on top of this, and to dockerized-deployments, which covers the broader Dev-Container-and-Docker philosophy this slots into.
>
> **NOTE. LAST VALIDATED: 2026-05.** Tool versions reflect current stable: Windows 11 23H2+, WSL kernel 5.15+, Ubuntu 24.04 LTS, NVIDIA driver R555+ on Windows, CUDA 12.4+ runtime, `nvidia-container-toolkit` current, PyTorch 2.x, JAX 0.4.x. Bump and re-validate annually per CLAUDE.md standard #5.

What this enables

- **Run CUDA-accelerated PyTorch, JAX, and TensorFlow** from a normal Ubuntu shell inside WSL2 — `import torch; torch.cuda.is_available() == True`.
- **Train and fine-tune models** on your local GPU instead of paying for a cloud instance for early-iteration work.
- **Compile and run custom CUDA kernels** via Numba, CuPy, Triton, and (if you really need it) `nvcc`.
- **Run GPU-accelerated containers** with `docker run --gpus all` from inside WSL — the same flag you'd use on a bare-metal Linux box.
- **Develop in Dev Containers with GPU access**, so a `Reopen in Container` action in VS Code drops you into an isolated environment that already has CUDA and the framework of your choice, with the host GPU passed through.

What this does NOT enable

- **Graphics rendering for display from inside WSL.** That's WSLg — a separate compositor stack that ships with WSL2. WSLg is fine for the occasional GUI app, but this doc is about **compute**, not display.
- **CUDA on Apple Silicon.** Different story. Apple Silicon Macs run MLX (or PyTorch's `mps` backend) on Apple's own GPU. There is no CUDA path on M-series chips and there isn't going to be one.
- **AMD ROCm on WSL.** AMD shipped ROCm for WSL in late 2024 and it has been improving, but the experience is still meaningfully rougher than NVIDIA's: fewer supported cards, narrower framework support, and more "compile from source" rabbit holes. This doc focuses on the NVIDIA path because that's what works smoothly today. If you have an AMD card, the high-level shape (Windows-side driver, WSL-side shim) is similar; the specifics aren't.
- **A magic-bullet performance win.** GPU paravirt in WSL2 is excellent (single-digit-percent overhead for compute), but if your code is bottlenecked on data loading off the Windows

filesystem, the GPU will sit idle. Keep your data on ext4 in `/home/julian/...`, not in `/mnt/c/...`. More on this in §14.

Prerequisites

- **Windows 10 21H2 or Windows 11** (Windows 11 strongly preferred — WSL improvements ship there first).
- **WSL2** installed and updated to a recent kernel (`wsl --update`).
- **Ubuntu 24.04 LTS** as your WSL distro. Other distros work; this doc is written for 24.04.
- **An NVIDIA GPU**, Pascal generation (GTX 10-series, 2016) or later. Turing or newer (RTX 20-series, 2018+) strongly recommended — those have tensor cores, which is where most of the ML perf actually comes from. As of 2026, RTX 30/40/50-series desktop or mobile is the assumed baseline.
- **A recent NVIDIA Windows driver** — R555 or newer. The Windows driver is what provides the WSL passthrough; the version is what gates which CUDA runtime versions work.
- **Docker working in WSL** — either Docker Desktop with WSL integration enabled, or `docker.io` installed natively in the distro. Required for the Dev Container path (steps 9-10). Not required for the bare WSL Python path (steps 7-8).
- ~30 minutes start-to-finish the first time; ~5 minutes if you already have WSL and the driver and just need to wire up the Python side.

Table of contents

---

1. The CUDA-on-WSL story

A quick history because the "why this works at all" is recent and load-bearing.

Pre-2020: not possible. Until mid-2020, GPUs in WSL did not exist. WSL2 had only just shipped (May 2020) with a real Linux kernel running in a lightweight Hyper-V VM, but the kernel had no path to talk to the host GPU. If you wanted CUDA from Linux on a Windows machine, you dual-booted.

June 2020: paravirt preview. NVIDIA and Microsoft jointly shipped the first preview of GPU paravirtualization for WSL2. The trick: instead of passing the entire GPU through to the WSL VM (which would have meant losing it on the Windows side, like a PCI passthrough on a server), they built `dxgkrnl`, a kernel module on the WSL side that proxies CUDA calls to the **Windows** driver. The Windows side keeps owning the hardware; WSL gets a virtual CUDA interface that's roughly as fast as native.

2021-2022: it got stable. The preview was rough. CUDA versions were limited, frameworks frequently broke. By mid-2022 the path was stable enough that PyTorch and TensorFlow officially supported it.

2023-2024: containers and frameworks settled in. The `nvidia-container-toolkit` (formerly `nvidia-docker2`) added WSL support, which meant `docker run --gpus all` worked inside WSL the same way it worked on bare-metal Linux. Docker Desktop automated the wiring. PyTorch, JAX, TensorFlow all shipped prebuilt wheels with CUDA bundled.

2025-2026: production-grade. As of 2026, GPU passthrough for WSL is a boring, working feature. The single biggest remaining cliff is **driver/runtime version skew** (Windows driver too old for the CUDA version a framework wheel was built against). That's the one thing this doc spends real time on.

Why this matters

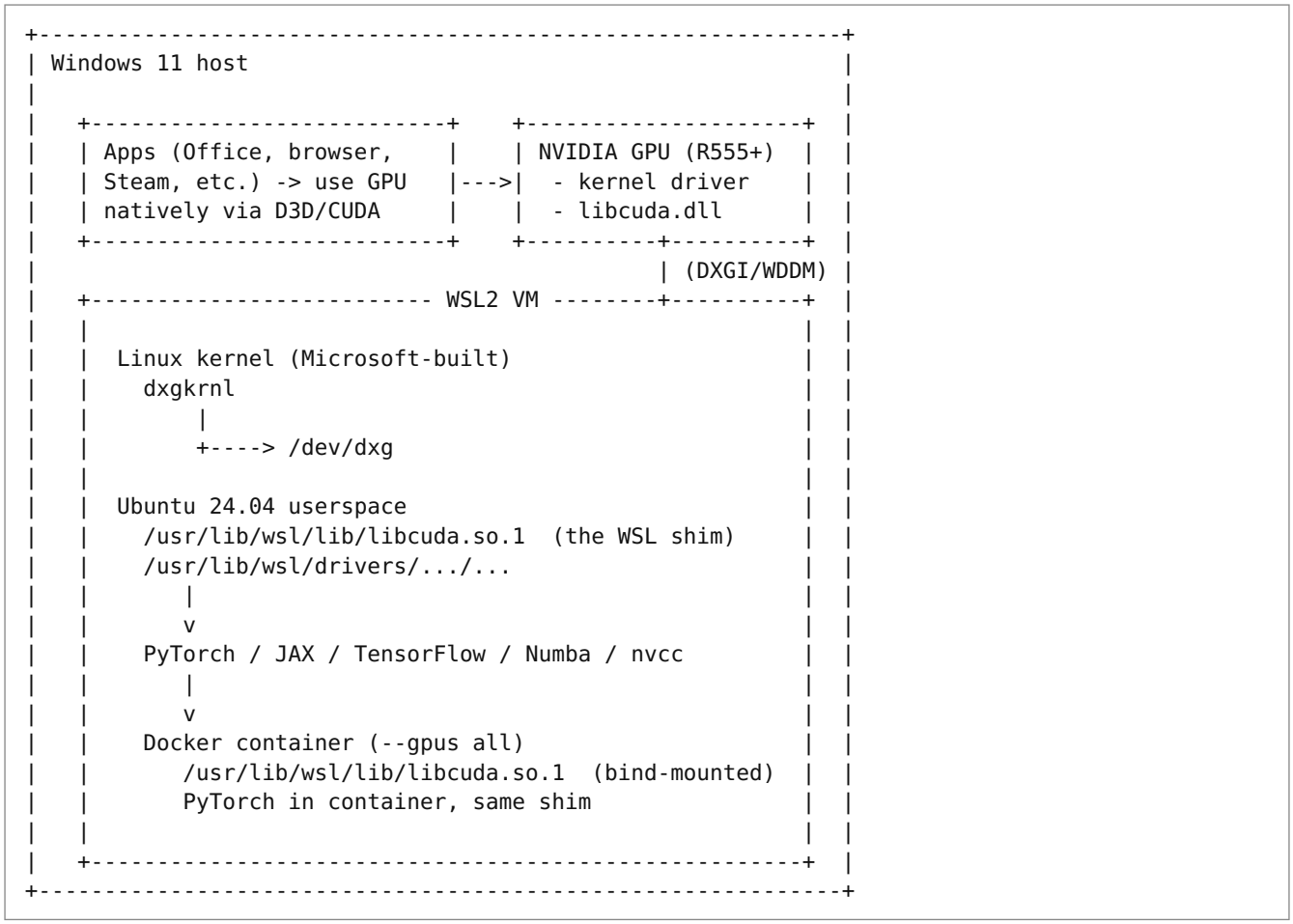
You can have one laptop that runs Windows for everything Windows is good at — Office, Teams, the school's proctored exam software, the airline's check-in app — and a real Linux ML workflow on the same hardware, sharing the GPU. No dual-boot reboots. No "I left my CUDA work on the Linux partition and now I need to sign a PDF in Adobe." It's the rare modern setup where you don't have to choose.

The flip side: when you'd reconsider

If you're doing **only** ML work and don't need Windows for anything, native Ubuntu on the bare metal is still a hair faster (no paravirt overhead, no double-OS resource cost) and meaningfully simpler to debug at the edges. The 1-5% perf gap is real but small; the operational simplicity is the bigger reason to dual-boot or wipe. See §14 for the perf numbers.

---

2. The architecture in one diagram (no Mermaid)



The three things to remember:

1. **The Windows NVIDIA driver owns the GPU.** It's the only driver. There is no Linux GPU driver in WSL, and you should never try to install one.
2. **`/usr/lib/wsl/lib/libcuda.so.1` is the WSL shim**, automatically mounted into your distro by WSL itself. It looks like a normal Linux CUDA library to userspace; under the hood it forwards calls to the Windows driver via `dxgkrnl` and `/dev/dxg`.
3. **Containers inherit the shim** when launched with `--gpus all`, courtesy of `nvidia-container-toolkit`. The container thinks it's talking to a real Linux CUDA library; the library forwards everything to the host.

---

3. Step 1: Install the NVIDIA Windows driver

The Windows driver provides everything. If it's missing or too old, nothing else works.

Why nvidia.com, not Windows Update

Windows Update will install an NVIDIA driver if you don't. It's usually 6-18 months out of date and sometimes ships drivers that have known WSL bugs that were fixed in newer releases. Use the official NVIDIA download.

Download and install

Go to nvidia.com/drivers. Pick:

- **Product type**: GeForce (for consumer cards) or RTX/Quadro (for workstation cards).
- **Product series**: Match your card.
- **Product**: Match exactly (e.g., "RTX 5070").
- **Operating system**: Windows 11.
- **Download type**: Game Ready Driver (GRD) or Studio Driver (SD). Either works for WSL CUDA. Studio is updated less often and tested for stability with creative apps; Game Ready is updated more often. Pick **Studio** for ML work — fewer driver bumps means fewer mystery WSL regressions.

Install with the default options. Reboot.

Verify on the Windows side

In PowerShell:

```
nvidia-smi
```

Expected output (truncated):

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 555.99                 Driver Version: 555.99         CUDA Version: 12.5     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                  TCC/WDDM    | Bus-Id        Disp.A  | Volatile Uncorr. ECC |
| ...                                     |                        |                      |
+-----------------------------------------+------------------------+----------------------+
```

The two numbers that matter:

- **Driver Version**: should be R555 or newer for CUDA 12.4 framework wheels.
- **CUDA Version**: this is the maximum CUDA runtime version this driver supports. It's not the toolkit installed; it's the ceiling. A driver advertising "CUDA Version: 12.5" can run any CUDA 12.x runtime ≤ 12.5.

> **IMPORTANT.** The "CUDA Version" in `nvidia-smi` is a ceiling, not a floor. If `nvidia-smi` says CUDA 12.5 and you install a framework wheel built for CUDA 12.4, it works. If you install a wheel built for CUDA 12.6 it does not. **WHEN IN DOUBT, RUN THE FRAMEWORK WHEEL THAT'S ONE MINOR BELOW WHAT THE DRIVER ADVERTISES.**

Failure modes here

Symptom	Cause	Fix
`nvidia-smi` not recognized in PowerShell	Driver install failed or didn't add to PATH	Re-run the installer; pick "Custom" → "Perform a clean installation"; reboot.
`nvidia-smi` works but reports an older driver than you installed	Old driver still resident; install didn't fully apply	"Clean install" option in the NVIDIA installer; or use Display Driver Uninstaller in Safe Mode, then reinstall.
`nvidia-smi` reports "No devices were found"	Card disabled in Device Manager, or PCIe slot not seated	Check Device Manager → Display adapters; on desktops, re-seat the card.

---

4. Step 2: Verify the GPU is visible in WSL

Open a WSL Ubuntu shell. **Do not install Linux NVIDIA drivers** — the Windows driver is the only driver. Ubuntu 24.04 already ships the WSL libcuda shim via the `wsl` package; you don't install anything on the Linux side to get `nvidia-smi`.

```
nvidia-smi
```

Expected output (this is on a desktop with one RTX 5070):

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 555.99                 Driver Version: 555.99         CUDA Version: 12.5     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name              Persistence-M    | Bus-Id        Disp.A   | Volatile Uncorr. ECC |
| Fan  Temp   Perf       Pwr:Usage/Cap    |          Memory-Usage  | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 5070        WDDM | 00000000:01:00.0  Off  |                  N/A |
|  0%   38C    P8             14W /  220W  |     1024MiB / 12288MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
```

The driver version and CUDA version reported here are **the same** as on the Windows side — because there's only one driver, and WSL is reading through to it.

What success looks like

- `nvidia-smi` runs without error.
- It reports the right card name (matches what you see in Device Manager on Windows).
- The driver and CUDA version match the Windows-side `nvidia-smi` output exactly.

Common failure modes

> **WARNING. "NVIDIA-SMI HAS FAILED BECAUSE IT COULDN'T COMMUNICATE WITH THE NVIDIA DRIVER."**
>
> Almost always one of three things:
>
> 1. **WSL KERNEL IS TOO OLD.** Run `wsl --update` from PowerShell. Restart WSL: `wsl --shutdown`, then reopen.
> 2. **WSL DISTRO IS TOO OLD.** If you're still on Ubuntu 20.04 or earlier, the libcuda shim isn't there. Either upgrade with `do-release-upgrade` or install a fresh Ubuntu 24.04 distro alongside.
> 3. **GPU COMPUTE MODE DISABLED / WINDOWS SIDE BROKEN.** Verify `nvidia-smi` works in PowerShell first. If it doesn't work there, fix it there before troubleshooting WSL.

```
Error: libcuda.so.1: cannot open shared object file: No such file or directory
```

This means a framework couldn't find the WSL shim. The shim lives at `/usr/lib/wsl/lib/libcuda.so.1`. Check it exists:

```
ls -l /usr/lib/wsl/lib/libcuda.so*
```

If missing, run `wsl --update` from PowerShell and restart WSL.

```
# Common belt-and-suspenders sanity check after the above.
ldconfig -p | grep libcuda
# Expect: libcuda.so.1 (libc6,x86-64) => /usr/lib/wsl/lib/libcuda.so.1
```

Stop here and proceed only if `nvidia-smi` works in WSL.

If it doesn't, no amount of `pip install torch` will save you. The rest of this doc assumes this step is green.

---

5. Step 3: CUDA toolkit (only if compiling kernels)

> **IMPORTANT. FOR 95% OF ML WORK, YOU DO NOT INSTALL THE CUDA TOOLKIT INSIDE WSL.** PyTorch, JAX, TensorFlow, CuPy, and most other major libraries ship prebuilt wheels with the relevant CUDA libraries (cuBLAS, cuDNN, NCCL, etc.) bundled inside. The wheel + the WSL shim is the entire CUDA "installation" you need. Skip ahead to §6.

The only times you actually need the CUDA toolkit (which gives you `nvcc`, the headers, and the unbundled libraries):

- You're writing CUDA C++ kernels and compiling them with `nvcc`.
- You're using a library that calls `nvcc` at install time (some Triton fork, some custom flash-attention build).
- You're profiling with Nsight Systems or Nsight Compute.
- You're integrating with a system that expects a "real" CUDA install on the system path.

When you DO need the toolkit: use the WSL-specific package

> **WARNING. DO NOT USE THE STANDARD LINUX CUDA INSTALLER (`CUDA-UBUNTU2404`, `CUDA_XX.X_LINUX.RUN`) INSIDE WSL.** Those packages bundle a Linux GPU driver and try to install it. That driver will overwrite the WSL libcuda shim and break GPU access. NVIDIA publishes a separate WSL-specific CUDA toolkit package that omits the driver.

Use this exact apt source — the `wsl-ubuntu` repo, not the regular `ubuntu2404` one:

```
# Add NVIDIA's WSL-specific apt repo.
wget https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64/cuda-
keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
rm cuda-keyring_1.1-1_all.deb

sudo apt update
sudo apt install -y cuda-toolkit-12-5
```

Note the package name: `cuda-toolkit-12-5`, **not** `cuda` (which would also install the driver) and **not** `nvidia-cuda-toolkit` (which is the Ubuntu-maintained, often outdated, version).

Verify:

```
/usr/local/cuda-12.5/bin/nvcc --version
# nvcc: NVIDIA (R) Cuda compiler driver
# Copyright (c) 2005-2024 NVIDIA Corporation
# Cuda compilation tools, release 12.5, V12.5.xxx
```

Optionally add to PATH in `~/.bashrc`:

```
export PATH=/usr/local/cuda-12.5/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.5/lib64:$LD_LIBRARY_PATH
```

Failure mode: you accidentally installed the wrong package

If after running `sudo apt install cuda` (without the `-toolkit-` suffix) you find `nvidia-smi` no longer works in WSL, you've installed the Linux GPU driver on top of the WSL shim. Recovery:

```
sudo apt remove --purge 'nvidia-*' 'cuda-drivers*' 'libnvidia-*'
sudo apt autoremove
# Then re-update WSL from PowerShell to restore the shim:
#   wsl --shutdown
#   wsl --update
```

Then verify `nvidia-smi` works again before doing anything else.

---

6. Step 4: PyTorch with CUDA, via uv

We use `uv` for everything Python-side. (See scientific-python-2026 for the full case, but the short version: `uv` replaces pip + venv + pip-tools + pyenv with one Rust binary that's ~50× faster.)

Create a project

```
mkdir -p ~/dev/gpu-sandbox && cd ~/dev/gpu-sandbox
uv init --python 3.12
```

Add PyTorch with the CUDA index

PyTorch wheels with CUDA bundled live on a PyTorch-hosted index, **not** PyPI. (PyPI's `torch` wheel is the CPU-only version — installing it without the index URL is the #1 reason people end up with `torch.cuda.is_available() == False`.)

```
uv add torch torchvision --index https://download.pytorch.org/whl/cu124
```

Pick the CUDA wheel index that matches your driver's ceiling, one minor below. As of 2026-05:

Driver advertises	Recommended wheel index
CUDA 12.5+	`https://download.pytorch.org/whl/cu124`
CUDA 12.4	`https://download.pytorch.org/whl/cu124`
CUDA 12.1-12.3	`https://download.pytorch.org/whl/cu121`
CUDA 11.8 (old card)	`https://download.pytorch.org/whl/cu118`

> **TIP.** The current list is at pytorch.org/get-started. PyTorch lags driver versions by ~6 months; if the driver is brand new there may not be a matching wheel yet. Use the highest available wheel `≤` the driver ceiling.

Verify

```
uv run python -c "
import torch
print('torch:', torch.__version__)
print('cuda available:', torch.cuda.is_available())
print('device count:', torch.cuda.device_count())
print('device 0:', torch.cuda.get_device_name(0))
print('capability:', torch.cuda.get_device_capability(0))
print('cuda runtime:', torch.version.cuda)
print('cudnn:', torch.backends.cudnn.version())
"
```

Expected (on an RTX 5070):

```
torch: 2.4.1+cu124
cuda available: True
device count: 1
device 0: NVIDIA GeForce RTX 5070
capability: (9, 0)
cuda runtime: 12.4
cudnn: 90100
```

Failure modes

> **WARNING. `TORCH.CUDA.IS_AVAILABLE()` RETURNS `FALSE`.**
>
> Three causes, in decreasing order of likelihood:
>
> 1. **YOU INSTALLED THE CPU WHEEL.** Check *`torch.__version__`* — if it's *`2.4.1`* (no *`+cu124`* suffix), you got CPU-only. Reinstall with the index URL: *`bash uv remove torch torchvision uv add torch torchvision --index https://download.pytorch.org/whl/cu124`*
> 2. **WHEEL CUDA VERSION > DRIVER CEILING.** Check both: *`nvidia-smi`* (the ceiling) and *`torch.version.cuda`* (the wheel's CUDA). If wheel > ceiling, downgrade either the wheel or update the Windows driver.
> 3. **`NVIDIA-SMI` ITSELF DOESN'T WORK IN WSL.** Back to §4. Don't chase Python errors until the system call works.

```
RuntimeError: CUDA error: no kernel image is available for execution on the device
```

The wheel was built without support for your GPU's compute capability. PyTorch wheels target a fixed set of compute capabilities; very new (e.g., compute 10.x on a brand-new card) or very old (compute 3.x) cards fall outside that set. Fix: use a nightly wheel, or build PyTorch from source with `TORCH_CUDA_ARCH_LIST` set.

---

7. Step 5: JAX with CUDA

JAX's GPU story is simpler than PyTorch's because JAX ships **one** wheel that auto-detects the CUDA situation and downloads the right `jax-cuda12-plugin` for it.

```
uv add "jax[cuda12]"
```

The `[cuda12]` extra pulls in `jax-cuda12-plugin` (the CUDA backend) and `jax-cuda12-pjrt` (the runtime). Both ship with bundled CUDA libraries, so you don't need anything on the system.

Verify

```
uv run python -c "
import jax
print('jax:', jax.__version__)
print('devices:', jax.devices())
print('default backend:', jax.default_backend())
import jax.numpy as jnp
x = jnp.ones((4, 4))
print('test matmul on', x.device, ':', (x @ x.T).sum())
"
```

Expected:

```
jax: 0.4.31
devices: [CudaDevice(id=0)]
default backend: gpu
test matmul on cuda:0 : 16.0
```

Failure modes

```
RuntimeError: Unable to initialize backend 'cuda': ...
```

Usually means the JAX CUDA plugin couldn't find a working `libcuda.so.1`. Same diagnostic chain as PyTorch — `nvidia-smi` first, then check `ldconfig -p | grep libcuda`. If both pass and JAX still can't find CUDA, set:

```
export JAX_PLATFORMS=cuda
```

and rerun. If JAX now says "no CUDA devices visible," it's the WSL shim — `wsl --update` from PowerShell.

```
W external/xla/xla/service/gpu/nvptx_compiler.cc: ...
```

A noisy warning that XLA can't find `ptxas` (part of the CUDA toolkit). JAX will still run — it falls back to a slower path. If you want the warning gone, install the CUDA toolkit per §5 and put `nvcc`'s directory on PATH so XLA can find `ptxas`.

---

8. Step 6: Docker GPU passthrough

Now the more interesting case: running CUDA workloads in containers from WSL. This is what makes the Dev Container path work.

Two paths, pick one

Setup	When to use	What to install
Docker Desktop with WSL integration enabled	You're using Docker Desktop already (most people on Windows are).	Nothing extra in WSL — Docker Desktop handles the GPU wiring automatically as of 4.20+.
`docker.io` installed natively in the WSL distro	You disabled Docker Desktop, or your workflow prefers Linux-native Docker.	`nvidia-container-toolkit`, manually configured.

The first path is dramatically simpler. **Use Docker Desktop unless you have a specific reason not to.** Docker Desktop's WSL integration installs and configures `nvidia-container-toolkit` inside the integrated distro for you.

> **NOTE.** Per CLAUDE.md, Docker Desktop's WSL integration may be disabled in your env. If it is, enable it (Docker Desktop → Settings → Resources → WSL Integration → toggle on for Ubuntu) or follow the manual install below.

Test it works

Either path, the same one-liner verifies GPU access from inside a container:

```
docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi
```

Expected output:

```
==========
== CUDA ==
==========

CUDA Version 12.4.0

[... library version banner ...]

+-----------------------------------------+-----------------------+----------------------+
| NVIDIA-SMI 555.99    Driver Version: 555.99    CUDA Version: 12.5                      |
+-----------------------------------------+-----------------------+----------------------+
|   0  NVIDIA GeForce RTX 5070        WDDM | 00000000:01:00.0  Off |                  N/A |
+-----------------------------------------+-----------------------+----------------------+
```

Same `nvidia-smi` output you'd get on the host — that means the container sees the GPU through the same shim.

Manual install: nvidia-container-toolkit

Only needed if you're using `docker.io` directly in WSL, not Docker Desktop.

```
# Add NVIDIA's container-toolkit repo.
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-
toolkit.list | \
  sed
's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt install -y nvidia-container-toolkit

# Configure the Docker daemon to use the NVIDIA runtime.
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Verify:

```
docker info | grep -i runtime
# Expect:
#  Runtimes: io.containerd.runc.v2 nvidia runc
#  Default Runtime: runc
```

Then the same `docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi` test.

Why `--gpus all` instead of `--runtime=nvidia`

The old way (pre-2020) was `--runtime=nvidia` plus `NVIDIA_VISIBLE_DEVICES=all`. Docker 19.03+ added `--gpus`, which is the modern equivalent and works without setting a default runtime. Use `--gpus all`; the old syntax still works but it's a sign of an old tutorial.

Failure modes

> **WARNING.** **`docker: Error response from daemon: could not select device driver "" with capabilities: [[gpu]].`**
> The toolkit isn't installed or the daemon isn't configured. Re-run *`sudo nvidia-ctk runtime configure --runtime=docker && sudo systemctl restart docker`*. If using Docker Desktop, toggle WSL integration off and back on.

```
nvidia-container-cli: initialization error: nvml error: driver/library version mismatch
```

Driver version mismatch between what the container expects and what the host shim provides. Update the Windows driver (`R555+`) and `wsl --update`, restart WSL.

```
docker: command not found
```

Docker Desktop WSL integration not enabled, and `docker.io` not installed in the distro. Pick one of the two paths and complete it.

---

9. Step 7: A Dev Container with GPU access

This is the canonical setup for new GPU projects: a `.devcontainer/` that gives you a fresh Ubuntu + CUDA + Python + your framework, with the host GPU passed through, in one `Reopen in Container` command.

Why bother with a container at all

Three reasons, in order of importance:

1. **Dev/prod parity.** The container that trains your model in dev is structurally close to one that could run it on a cloud GPU box later. No "I trained it on my laptop and now the deps don't resolve on Lambda Labs."
2. **Reproducibility across reinstalls.** Wipe your WSL distro and rebuild. The container is recreated from `Dockerfile` + `devcontainer.json` + lockfile. Nothing in `~` matters except the code itself.
3. **Multiple projects with different CUDA versions.** Project A uses CUDA 12.4 + PyTorch 2.4; project B is stuck on CUDA 11.8 + PyTorch 1.13 because of a finicky dep. Different containers, no system-wide CUDA installs fighting each other.

`.devcontainer/devcontainer.json`

```
{
  "name": "gpu-sandbox",
  "build": { "dockerfile": "Dockerfile" },
  "runArgs": [
    "--gpus=all",
    "--shm-size=8g"
  ],
  "containerEnv": {
    "NVIDIA_VISIBLE_DEVICES": "all",
    "NVIDIA_DRIVER_CAPABILITIES": "compute,utility"
  },
  "mounts": [
    "source=${localEnv:HOME}/.cache/uv,target=/root/.cache/uv,type=bind,consistency=cached"
  ],
  "customizations": {
    "vscode": {
      "extensions": [
        "ms-python.python",
        "ms-python.vscode-pylance",
        "charliermarsh.ruff",
        "ms-toolsai.jupyter",
        "ms-azuretools.vscode-docker",
        "nvidia.nsight-vscode-edition"
      ],
      "settings": {
        "python.defaultInterpreterPath": "/workspace/.venv/bin/python"
      }
    }
  },
  "postCreateCommand": "uv sync --frozen"
}
```

Field-by-field:

- **`runArgs: ["--gpus=all"]`** — the load-bearing line. This is what tells Docker to pass through every visible GPU. Without it, the container has no GPU access regardless of what's installed inside.
- **`--shm-size=8g`** — PyTorch DataLoader uses `/dev/shm` for inter-process memory transfer when `num_workers > 0`. The default `64m` overflows on real datasets and produces cryptic "Bus error" crashes. Set this to roughly your RAM / 4 for headroom.
- **`NVIDIA_VISIBLE_DEVICES`** / **`NVIDIA_DRIVER_CAPABILITIES`** — used by `nvidia-container-toolkit` to decide what's exposed. `compute,utility` is the minimum for ML; add `video` if you need NVENC for video encoding.
- **`mounts: uv cache`** — speeds up `uv sync` by sharing the wheel cache with the host. Skip for fully reproducible builds; include for fast rebuilds.
- **`postCreateCommand: uv sync --frozen`** — runs once on container creation. Installs all deps from `uv.lock`.

`.devcontainer/Dockerfile`

```
# NVIDIA's official base image — Ubuntu 22.04 + CUDA 12.4 + cuDNN.
# Pin the minor version (12.4.0) so the image hash is stable for a project lifetime.
# Bump the pin when bumping the rest of the stack, not on every build.
FROM nvidia/cuda:12.4.0-devel-ubuntu22.04

# System deps. -devel image has nvcc and headers; we add python, git, curl, plus
# build essentials that some ML packages still need for source builds.
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
        python3.12 python3.12-venv python3-pip \
        git curl ca-certificates build-essential \
        ffmpeg libsm6 libxext6 \
    && rm -rf /var/lib/apt/lists/*

# Install uv (the same as on the host; see scientific-python-2026).
RUN curl -LsSf https://astral.sh/uv/install.sh | sh
ENV PATH="/root/.local/bin:${PATH}"

# Pre-create a workspace dir; the devcontainer will bind-mount the repo into it.
WORKDIR /workspace

# Default to interactive bash.
CMD ["bash"]
```

Why this and not `python:3.12-slim`:

- `python:3.12-slim` has no CUDA. You'd install CUDA on top, which means dragging in the entire toolkit (~3GB) — the `nvidia/cuda:*-devel` image is what NVIDIA recommends, ships smaller, and is tested against current drivers.
- `nvidia/cuda:*-base` is smaller but lacks `nvcc` and headers. Pick `devel` if you might compile anything; `runtime` if you definitely won't. **Default to `devel`** — the size difference is ~1GB and you'll thank yourself the first time a package needs to JIT-compile something.
- `cuda:12.4.0-devel-ubuntu22.04` (not 24.04) — as of 2026-05, NVIDIA's official tag matrix still has 22.04 as the most-tested base. They publish 24.04 images too; once they hit "official" for a major CUDA version, swap to it.

`pyproject.toml` (the framework half)

```
[project]
name = "gpu-sandbox"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "torch",
    "torchvision",
    "jax[cuda12]",
    "numpy",
    "polars",
    "matplotlib",
    "tqdm",
]

[tool.uv.sources]
torch = { index = "pytorch-cu124" }
torchvision = { index = "pytorch-cu124" }

[[tool.uv.index]]
name = "pytorch-cu124"
url = "https://download.pytorch.org/whl/cu124"
explicit = true
```

The `[tool.uv.sources]` + `[[tool.uv.index]]` block is `uv`'s way of saying "pull `torch` from the PyTorch CUDA index, everything else from PyPI." Without it, `uv add torch` reaches for PyPI and you get the CPU wheel by accident. With it, `uv sync` is fully reproducible — including the CUDA wheels.

Spinning it up

In VS Code, with the **Dev Containers** extension installed:

1. `code ~/dev/gpu-sandbox`
2. Command Palette → "Dev Containers: Reopen in Container"
3. Wait for the first build (~3-5 minutes).
4. Open a terminal in VS Code — it's now inside the container.
5. Run the verification snippets from §6 and §7. Both should report GPU available.

What this gets you

- A fresh Ubuntu environment per project, with CUDA preinstalled, GPU passed through, the right Python, your framework of choice, your IDE extensions.
- Disposable — destroy the container, the host is untouched.
- Reproducible — anyone with the repo and Docker + an NVIDIA GPU can `Reopen in Container` and have the same setup.

---

10. Step 8: End-to-end verification

A 30-line PyTorch script that exercises the full stack: forward pass, backward pass, optimizer step, on the GPU. Run this in both the bare WSL Python env and the Dev Container env. Both should produce roughly the same timing.

Save as `verify_gpu.py`:

```
"""End-to-end GPU verification: a tiny transformer block trained for 50 steps.

Success criterion: completes without error, reports a device that is not "cpu",
and loss decreases over the run.
"""
import time

import torch
import torch.nn as nn

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Device: {device}")
if device == "cpu":
    raise SystemExit("CUDA not available — this is not the test we wanted.")

print(f"GPU: {torch.cuda.get_device_name(0)}")
print(f"CUDA runtime: {torch.version.cuda}")
print(f"cuDNN: {torch.backends.cudnn.version()}")

torch.manual_seed(0)

# A tiny transformer: 4 layers, 256-dim, 8 heads. ~3M params.
model = nn.TransformerEncoder(
    nn.TransformerEncoderLayer(d_model=256, nhead=8, dim_feedforward=1024, batch_first=True),
    num_layers=4,
).to(device)
head = nn.Linear(256, 10).to(device)

opt = torch.optim.AdamW(list(model.parameters()) + list(head.parameters()), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

batch, seqlen, dim = 64, 128, 256

print(f"\nTraining for 50 steps on synthetic data ({batch=}, {seqlen=}, {dim=})...")

torch.cuda.synchronize()
start = time.perf_counter()

for step in range(50):
    x = torch.randn(batch, seqlen, dim, device=device)
    y = torch.randint(0, 10, (batch,), device=device)
    logits = head(model(x).mean(dim=1))
    loss = loss_fn(logits, y)
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 10 == 0:
        print(f"  step {step:3d}  loss={loss.item():.4f}")

torch.cuda.synchronize()
elapsed = time.perf_counter() - start
print(f"\nDone in {elapsed:.2f}s ({50 / elapsed:.1f} steps/sec).")
print(f"Peak GPU memory: {torch.cuda.max_memory_allocated() / 1e9:.2f} GB")
```

Run it:

```
uv run python verify_gpu.py
```

Expected timings

Order of magnitude only — exact numbers vary with PyTorch version, kernel fusion, etc.

GPU	50 steps	steps/sec
Desktop RTX 5070	~1.8s	~28
Desktop RTX 5060	~2.5s	~20
Laptop RTX 4060 mobile	~3.5s	~14
Laptop RTX 5070 mobile	~2.3s	~22
CPU only (Ryzen 7 / i7)	~45-60s	~1

If you're seeing CPU-tier numbers despite `torch.cuda.is_available() == True`, the model or data isn't actually on GPU — search the code for missing `.to(device)` calls.

Add this as a project sanity check

For new GPU projects, drop this script in `scripts/verify_gpu.py` and run it as part of the dev container's `postCreateCommand`. If the script ever stops passing, the container is broken before you've written any code — much easier to debug than discovering it three hours into a debugging session at midnight.

---

11. Multi-GPU notes

Rare in WSL setups (most laptops have one GPU, and dual-GPU desktops with both used for compute are uncommon outside of dedicated workstations), but the basics:

Inventory

```
nvidia-smi -L
# GPU 0: NVIDIA GeForce RTX 5090 (UUID: GPU-...)
# GPU 1: NVIDIA GeForce RTX 5070 (UUID: GPU-...)
```

Restrict visibility

```
# Use only GPU 0.
CUDA_VISIBLE_DEVICES=0 uv run python verify_gpu.py

# Use GPUs 0 and 1, in that order (so torch.device("cuda:0") = physical GPU 0).
CUDA_VISIBLE_DEVICES=0,1 uv run python verify_gpu.py

# Use no GPU even if CUDA is built in.
CUDA_VISIBLE_DEVICES="" uv run python verify_gpu.py
```

Pin a process in code

```
import torch
device = torch.device("cuda:1")  # or "cuda:0"
model = model.to(device)
```

Docker multi-GPU

```
# All GPUs (default).
docker run --rm --gpus all ...

# Just GPU 1.
docker run --rm --gpus '"device=1"' ...

# Two specific GPUs.
docker run --rm --gpus '"device=0,1"' ...
```

The doubled quoting is because Docker parses `--gpus` as a JSON-like spec; quotes inside are mandatory.

When you actually have multi-GPU on WSL

You almost certainly want `torch.distributed` with NCCL backend or DDP via `torchrun`. That's a bigger topic than this doc covers — but the WSL-specific gotcha is that NCCL's peer-to-peer transport (NVLink) is **not** exposed through paravirt. On WSL with two GPUs, NCCL falls back to PCIe-through-host transport, which is slower than NVLink. For genuine multi-GPU production training, bare-metal Linux is meaningfully better.

---

12. Common issues and diagnostics

Symptoms-to-causes table first, then the longer treatments. Use this as a checklist.

Symptom	First place to look
`nvidia-smi` not found in WSL	WSL kernel not updated; `wsl --update` from PowerShell
`nvidia-smi` runs but reports a stale driver	Windows-side `nvidia-smi` first — fix there before WSL
`torch.cuda.is_available() == False`	Did you `uv add torch` without the `--index` flag? (CPU wheel)
`RuntimeError: no kernel image is available`	GPU compute capability not in the wheel; use nightly or build from source
`CUDA error: out of memory`	Smaller batch, mixed-precision, gradient checkpointing
`docker: Error response from daemon: could not select device driver "" with capabilities: [[gpu]]`	Toolkit missing or daemon not configured
`nvidia-container-cli: initialization error: driver/library version mismatch`	Windows driver update + `wsl --update`, restart WSL
"No CUDA-capable device is detected" after sleep	Restart WSL: `wsl --shutdown` from PowerShell
Slow training despite `cuda` device	Data loader bottleneck — keep data in `/home/julian`, not `/mnt/c`
Mystery hangs on first CUDA call	Antivirus / EDR scanning libcuda; whitelist `\\wsl$\Ubuntu\usr\lib\wsl\`

Driver version mismatch

The single most common source of pain. Three numbers have to agree:

1. **The Windows NVIDIA driver version.**
2. **The CUDA "ceiling" advertised by `nvidia-smi`.**
3. **The CUDA version the framework wheel was built against** (`torch.version.cuda`, or for TF: `tf.sysconfig.get_build_info()["cuda_version"]`).

Rule: **(3) ≤ (2).** If your driver advertises CUDA 12.5 and your wheel is for CUDA 12.6, the wheel fails to load. Always pick a wheel for a CUDA version ≤ what `nvidia-smi` advertises.

Recovery: bump (1) and (2) by updating the Windows driver, or downgrade (3) by switching to an older wheel index.

Out of memory (OOM)

```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 1.21 GiB.
GPU 0 has a total capacity of 12.00 GiB of which 1.07 GiB is free.
```

In order of escalating effort:

```
# Watch live GPU usage in WSL.
nvidia-smi -l 1

# Or with a fancier UI:
sudo apt install nvtop
nvtop
```

```
# 1. Reduce batch size — almost always the right first try.
batch_size = 32  # was 128

# 2. Enable mixed precision — halves memory for activations.
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()
with autocast(dtype=torch.bfloat16):
    loss = model(x)
scaler.scale(loss).backward()
scaler.step(opt)
scaler.update()

# 3. Gradient checkpointing — trades compute for memory.
from torch.utils.checkpoint import checkpoint_sequential
out = checkpoint_sequential(model_layers, segments=4, input=x, use_reentrant=False)

# 4. Move optimizer state off GPU (for very large models).
opt = bitsandbytes.optim.AdamW8bit(model.parameters())  # 75% reduction in optimizer state
```

Per-step OOM also sometimes means a memory leak — tensors are being kept alive that shouldn't be. Check for:

- Accumulating loss / metrics tensors without `.item()` or `.detach()`.
- Storing intermediate activations in a Python list.
- Validation loop without `torch.no_grad()` or `torch.inference_mode()`.

WSL kernel out of date

```
W tensorflow/core/common_runtime/gpu/gpu_device.cc: Could not load dynamic library
'libcudart.so.12'
```

or

```
nvidia-smi: command not found
```

Both can be the WSL kernel being too old. From PowerShell **on the Windows host** (not inside WSL):

```
wsl --update
wsl --shutdown
```

Then reopen WSL. `wsl --update` will pull the latest Microsoft-built kernel, which is where `dxgkrnl` lives.

Docker Desktop integration not enabled

Symptoms: `docker` not on PATH in WSL, or `docker run --gpus all` fails with "could not select device driver."

Fix: Docker Desktop → Settings → Resources → WSL Integration → toggle **Enable integration with my default WSL distro** and check the specific Ubuntu distro. Apply & Restart Docker Desktop. Reopen WSL.

Antivirus or EDR blocking libcuda loading

Less common, but real on managed corporate laptops. Symptoms: framework hangs on first CUDA call, or `nvidia-smi` works but framework can't see the GPU.

Diagnose: run with `strace -f -e openat python verify_gpu.py 2>&1 | grep libcuda` and look for `EACCES` / `EPERM` near the libcuda open. If you see those, the AV is intercepting the file open.

Fix (if you control the AV): whitelist `\\wsl.localhost\Ubuntu\usr\lib\wsl\` on the Windows side. If you don't control it, there's no clean workaround — escalate to whoever does, or use dual-boot Linux.

"No CUDA-capable device is detected" after sleep

```
RuntimeError: No CUDA GPUs are available
```

Specifically after the laptop has slept. WSL2's virtual hardware state doesn't always survive a sleep/resume cycle cleanly.

Fix:

```
# From PowerShell on the host.
wsl --shutdown
```

Then reopen WSL. The GPU re-appears.

You can also work around this preemptively by **disabling sleep on the laptop while training**: Windows → Settings → System → Power → "Sleep" → "Never" while on AC power. Trade-off is the obvious one.

---

13. Performance notes

Overhead vs native Linux

Empirically (PyTorch microbenchmarks, ResNet-50 training, transformer fine-tuning): **WSL2 GPU paravirt costs <5% compared to bare-metal Linux on the same hardware**, for pure-compute workloads. cuBLAS, cuDNN, custom kernels — they all run at full speed; the paravirt overhead is in the IOCTL dispatch for kernel launches, which is microseconds and rounds to noise on any non-trivial kernel.

The cases where overhead is bigger:

- **Kernel-launch-bound workloads** (many tiny kernels): the per-launch dispatch overhead is measurable. 5-15%. Fix: kernel fusion (`torch.compile`), bigger batches.

- **Host-to-GPU transfers** of small tensors: similar story. The fix is to do fewer, larger transfers, which you should be doing anyway.
- **Multi-GPU with NVLink**: paravirt doesn't expose NVLink, so multi-GPU bandwidth drops to PCIe. 30-50% in some collective-heavy workloads. Bare-metal Linux is the fix.

For single-GPU training of any reasonably-sized model, you will not see a meaningful gap.

The data-loader trap

> **IMPORTANT. THE SINGLE BIGGEST CAUSE OF "WSL FEELS SLOW" IS DATA LOADED FROM `/MNT/C`, NOT GPU PARAVIRT OVERHEAD.** The 9p mount Windows uses to expose `C:` to WSL is 10-100× slower than ext4 for random-access reads — which is the access pattern of most DataLoader workloads.

Fix: keep your data on ext4. Symlink if needed.

```
# Bad: training reads each image off /mnt/c via 9p.
data_dir = "/mnt/c/Users/joshu/datasets/imagenet"

# Good: data lives in WSL's ext4 partition.
data_dir = "/home/julian/datasets/imagenet"

# Acceptable middle ground: symlink, but the underlying read is still 9p — only useful if the
# Windows side is the canonical store (e.g., it's syncing to OneDrive).
ln -s "/mnt/c/Users/joshu/datasets/imagenet" ~/datasets/imagenet
```

A 1000-image-per-second DataLoader on ext4 can drop to 50-100 images/sec on `/mnt/c`. That looks like "my GPU is at 5% utilization" — and the fix is filesystem, not GPU.

Mixed precision is free perf

Modern NVIDIA GPUs (Volta and newer) have tensor cores that run BF16/FP16 matmul at 2-8× the speed of FP32. Turning on autocast is almost always a win for training; the loss numerically is minimal.

```
from torch.cuda.amp import autocast
with autocast(dtype=torch.bfloat16):
    loss = model(x)
loss.backward()
```

BF16 (Brain Float 16) is preferred over FP16 for most training because its exponent range matches FP32. Use FP16 only if you're stuck on Volta (V100), which has FP16 tensor cores but not BF16.

`torch.compile` if you're on PyTorch 2.x

Wrap your model:

```
model = torch.compile(model, mode="reduce-overhead")
```

First call is slow (the compile pass), subsequent calls are 20-50% faster on most workloads. The win comes from kernel fusion — `torch.compile` traces the model and emits fused kernels via Triton, which is where most of WSL's small-kernel overhead gets eliminated anyway.

Power and thermals

Laptops throttle. A laptop 4060 mobile on battery can run at 35W; on AC at 115W. The difference is not 3× perf — but it is 2×. Always train on AC. If sustained training matters, look at the laptop's "performance mode" / "MaxQ" toggle.

```
# Check current power draw and thermal state from WSL.
nvidia-smi --query-gpu=power.draw,temperature.gpu,clocks.sm --format=csv -l 5
```

If clocks are bouncing around significantly and temp is at the GPU's hot limit (usually ~83°C), you're thermally throttled. Better cooling pad, lower ambient temp, or reduce load.

---

14. Alternatives considered

Path	Verdict	Why
WSL2 GPU passthrough (this guide)	The default for "I have Windows and an NVIDIA GPU."	Lets you keep the Windows ecosystem and have a real Linux ML workflow; near-native perf.
Native Linux dual-boot	Use when ML is the primary use of the machine.	1-5% faster, simpler debugging at edges, real NVLink. Pays the cost of "must reboot to do Office."
Cloud GPU (Modal, RunPod, Lambda, vast.ai)	Use when local GPU isn't enough, or when you need A100/H100-class hardware.	Pay-per-use; access to bigger cards than any laptop. Operationally heavier; data egress costs.
Colab / Kaggle notebooks	Use for small experiments, throwaway exploration, sharing reproducibles.	Free tiers exist; UI overhead and ephemeral storage are the costs.
Apple Silicon + MLX or PyTorch MPS	Use if you bought into the Mac ecosystem and have no NVIDIA card.	Genuinely fast for inference; training is fine for smaller models; not CUDA — no compatibility with most CUDA-only libs (Triton, flash-attn).
AMD ROCm on WSL	Use only if you specifically have an AMD card and can't switch.	Improving but rougher than NVIDIA; fewer supported cards; some frameworks not fully there.
Docker Desktop's "Compose with GPU" without WSL	Use only on a Linux host with no WSL involved.	Not relevant if you're reading this on a Windows machine.

When to switch from WSL to dual-boot

The decision criterion isn't "is my GPU work serious?" — it's "do I still get value from Windows?" If the answer is yes, stay on WSL. If you find yourself rebooting from a Linux live USB to do ML because WSL is in your way (NVLink, weird multi-GPU drivers, kernel-mode features), dual-boot.

When to switch from local GPU to cloud

The math is roughly:

- A used desktop RTX 3090 (24GB) is ~$700, ~$0.02/hr amortized over 3 years at 8hrs/day usage.
- An RTX 4090 (24GB) on RunPod is ~$0.40/hr.
- An H100 (80GB) on Lambda Labs is ~$2.50/hr.

For day-to-day learning and experimentation on models that fit in 12-24GB, local wins by 10-50×. For one-off big training runs that need 40-80GB or many GPUs, cloud wins on capability if not on per-hour cost.

The hybrid pattern: **prototype locally, train at scale in cloud.** A Dev Container that runs on your laptop and on a cloud GPU box is the bridge. Same `pyproject.toml`, same `Dockerfile`, same code.

---

15. Cheat sheet

Print this. Pin it.

Setup verification (run in this order)

```
# 1. Windows side: PowerShell.
nvidia-smi                                    # Driver R555+, CUDA 12.5+ ceiling

# 2. WSL side.
nvidia-smi                                    # Same output as Windows
ls /usr/lib/wsl/lib/libcuda.so*               # Shim present
ldconfig -p | grep libcuda                    # Linker finds it

# 3. Python side (with uv project initialized).
uv run python -c "import torch; print(torch.cuda.is_available(),
torch.cuda.get_device_name(0))"

# 4. Docker side.
docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi
```

Recovery / nuking from orbit

```
# From PowerShell on the host.
wsl --update
wsl --shutdown          # then reopen any WSL terminal

# Inside WSL: did you accidentally install Linux NVIDIA drivers?
sudo apt remove --purge 'nvidia-*' 'cuda-drivers*' 'libnvidia-*'
sudo apt autoremove
# Then wsl --shutdown && wsl --update from PowerShell.
```

Important CUDA paths in WSL

```
/usr/lib/wsl/lib/libcuda.so.1           # The WSL shim (DO NOT delete)
/usr/lib/wsl/drivers/                   # Driver files mounted from Windows
/dev/dxg                                # The kernel paravirt device
/usr/local/cuda-12.5/bin/nvcc           # nvcc (if you installed the toolkit, §5)
/usr/local/cuda-12.5/lib64/             # Toolkit libs (if installed)
```

`nvidia-smi` essentials

```
nvidia-smi                                        # One-shot snapshot
nvidia-smi -l 1                                   # Live, refresh 1s
nvidia-smi -L                                     # List GPUs by index + UUID
nvidia-smi --query-gpu=name,utilization.gpu,memory.used,memory.total --format=csv
nvidia-smi --query-gpu=power.draw,temperature.gpu --format=csv -l 5
nvidia-smi pmon -c 1                              # Per-process GPU usage
nvtop                                             # Better TUI (apt install nvtop)
```

Docker GPU one-liners

```
# All GPUs.
docker run --rm --gpus all <image> <cmd>

# Specific GPU(s).
docker run --rm --gpus '"device=0"' <image> <cmd>
docker run --rm --gpus '"device=0,1"' <image> <cmd>

# Common smoke test.
docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi

# With shared memory (PyTorch DataLoader).
docker run --rm --gpus all --shm-size=8g <image> <cmd>

# Compose equivalent.
# services:
#   trainer:
#     image: my-trainer
#     deploy:
#       resources:
#         reservations:
#           devices:
#             - driver: nvidia
#               count: all
#               capabilities: [gpu]
```

Framework device-info one-liners

```
# PyTorch.
uv run python -c "import torch; print(torch.cuda.is_available(),
torch.cuda.get_device_name(0), torch.version.cuda)"

# JAX.
uv run python -c "import jax; print(jax.devices(), jax.default_backend())"

# TensorFlow.
uv run python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"

# CuPy.
uv run python -c "import cupy; print(cupy.cuda.runtime.getDeviceCount(),
cupy.cuda.runtime.getDeviceProperties(0)['name'])"

# Numba.
uv run python -c "from numba import cuda; print(cuda.is_available(),
cuda.get_current_device().name)"
```

Choosing the PyTorch wheel index

```
# Match the LOWER of (Windows driver CUDA ceiling, latest PyTorch CUDA wheel).
# As of 2026-05:
uv add torch --index https://download.pytorch.org/whl/cu124   # newest, ≤ R555 driver
uv add torch --index https://download.pytorch.org/whl/cu121   # for older drivers
uv add torch --index https://download.pytorch.org/whl/cu118   # for Pascal cards / very old
drivers
uv add torch --index https://download.pytorch.org/whl/cpu     # explicit CPU-only
```

JAX install

```
uv add "jax[cuda12]"          # CUDA 12.x — the default for modern setups
uv add "jax[cuda11_pip]"      # only if forced onto CUDA 11 by old driver
```

Memory monitoring while training

```
# Terminal 1: training.
uv run python train.py

# Terminal 2: live watch.
nvtop                     # best
# or
watch -n 1 nvidia-smi     # plain

# Or in-Python:
python -c "import torch; print(f'{torch.cuda.memory_allocated()/1e9:.2f} GB')"
```

`CUDA_VISIBLE_DEVICES`

```
CUDA_VISIBLE_DEVICES=0 uv run python ...    # only GPU 0
CUDA_VISIBLE_DEVICES=1 uv run python ...    # only GPU 1
CUDA_VISIBLE_DEVICES=0,1 uv run python ...  # both, GPU 0 = cuda:0
CUDA_VISIBLE_DEVICES="" uv run python ...   # force CPU
```

Dev Container quick reference

```
// .devcontainer/devcontainer.json — the GPU-relevant fields.
{
  "runArgs": ["--gpus=all", "--shm-size=8g"],
  "containerEnv": {
    "NVIDIA_VISIBLE_DEVICES": "all",
    "NVIDIA_DRIVER_CAPABILITIES": "compute,utility"
  }
}
```

```
# .devcontainer/Dockerfile — the GPU-relevant lines.
FROM nvidia/cuda:12.4.0-devel-ubuntu22.04
# ... your Python + uv layers ...
```

The "is my GPU actually being used?" checklist

- [ ] `nvidia-smi` in WSL shows the right card and driver.
- [ ] `torch.cuda.is_available()` returns `True`.
- [ ] Inside the training loop, `next(model.parameters()).device` reports `cuda:0`.
- [ ] During training, `nvidia-smi -l 1` shows GPU utilization > 20% sustained (lower means data-loader-bound).
- [ ] Memory usage in `nvidia-smi` increases when the model loads (proves weights are on GPU).
- [ ] CPU is < 100% per core (if it's at 100%, you're CPU-bound on data loading — fix the pipeline).
- [ ] Data is read from `/home/julian/...`, not `/mnt/c/...`.
- [ ] You're not running in a CPU-fallback path because of a missing kernel for your compute capability.

When something stops working: order of bisection

1. **Windows-side** `nvidia-smi` — driver alive?
2. **WSL-side** `nvidia-smi` — passthrough alive?
3. `ls /usr/lib/wsl/lib/libcuda.so*` — shim present?
4. `docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi` — container layer alive?
5. `uv run python -c "import torch; torch.cuda.is_available()"` — framework sees it?
6. **The actual training script.**

Each layer succeeding tells you the next layer is the one to debug. Each layer failing tells you to fix it before moving on.

Cross-references

- scientific-python-2026 — the Python data stack that sits on top of this (PyTorch, JAX, Polars, Marimo, uv).
- dockerized-deployments — the Dev Container philosophy and the CI/CD pipeline that takes a GPU-trained model from laptop to a VPS (CPU inference) or cloud (GPU inference).
- vps-from-zero — the destination box, when you take a trained model from local GPU work to a hosted service.